

Sneha Verma

Test ID: 450017180566700 | 9810680996 | sneha.verma.cs27@iilm.edu

Test Date: April 1, 2026

Computer Science 75 /100	Logical Ability 69 /100	Computer Programming 51 /100	Quantitative Ability (Advanced) 58 /100
English Comprehension 77 /100	WriteX - Essay Writing 86 /100	Automata 24 /100	Automata Fix 15 /100
Personality Completed			

Computer Science			75 / 100
OS and Computer Architecture	DBMS	Computer Networks	
64 / 100	80 / 100	100 / 100	

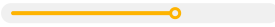
Logical Ability			69 / 100
Inductive Reasoning	Deductive Reasoning	Abductive Reasoning	
70 / 100	71 / 100	66 / 100	

Computer Programming			51 / 100
Basic Programming	Data Structures	OOP and Complexity Theory	
47 / 100	58 / 100	47 / 100	

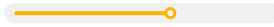
Quantitative Ability (Advanced)

 58 / 100

Basic Mathematics

 62 / 100

Advanced Mathematics

 59 / 100

Applied Mathematics

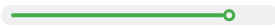
 54 / 100

English Comprehension

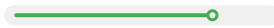
 77 / 100

CEFR: C1

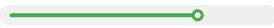
Grammar

 83 / 100

Vocabulary

 76 / 100

Comprehension

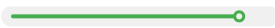
 71 / 100

WriteX - Essay Writing

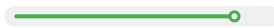
 86 / 100

CEFR: C1

Content Score

 87 / 100

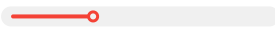
Grammar Score

 84 / 100

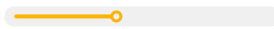
Automata

 24 / 100

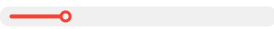
Programming Ability

 30 / 100

Programming Practices

 38 / 100

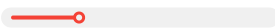
Functional Correctness

 20 / 100

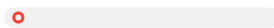
Automata Fix

 15 / 100

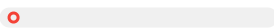
Logical Error

 25 / 100

Code Reuse

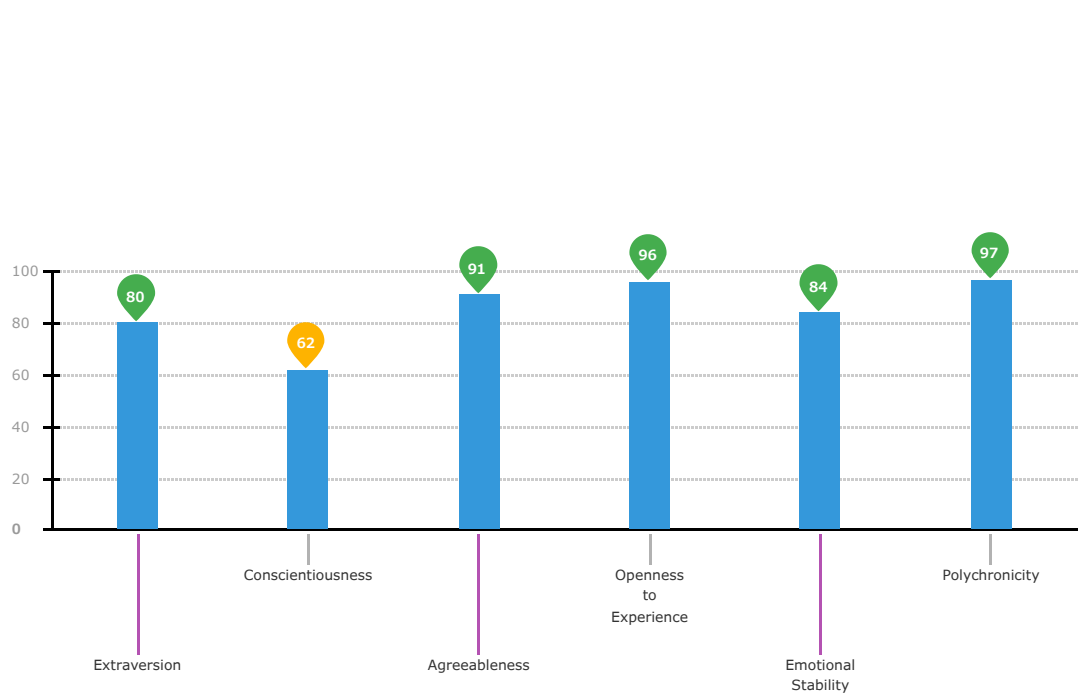
 0 / 100

Syntactical Error

 0 / 100

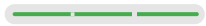
Personality

Completed

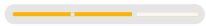


Competencies

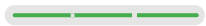
People Interaction



Self-Drive



Trainability



Repetitive Job Suitability



Work attributes

1 Introduction

About the Report

This report provides a detailed analysis of the candidate's performance on different assessments. The tests for this job role were decided based on job analysis, O*Net taxonomy mapping and/or criterion validity studies. The candidate's responses to these tests help construct a profile that reflects her/his likely performance level and achievement potential in the job role

This report has the following sections:

The **Summary** section provides an overall snapshot of the candidate's performance. It includes a graphical representation of the test scores and the subsection scores.

The **Insights** section provides detailed feedback on the candidate's performance in each of the tests. The descriptive feedback includes the competency definitions, the topics covered in the test, and a note on the level of the candidate's performance.

The **Response** section captures the response provided by the candidate. This section includes only those tests that require a subjective input from the candidate and are scored based on artificial intelligence and machine learning.

The **Learning Resources** section provides online and offline resources to improve the candidate's knowledge, abilities, and skills in the different areas on which s/he was evaluated.

Score Interpretation

All the test scores are on a scale of 0-100. All the tests except personality and behavioural evaluation provide absolute scores. The personality and behavioural tests provide a norm-referenced score and hence, are percentile scores. Throughout the report, the colour codes used are as follows:

- $70 \leq \text{Score} < 100$
- $30 \leq \text{Score} < 70$
- $0 \leq \text{Score} < 30$

2 | Insights

English Comprehension

 77 / 100CEFR: **C1**

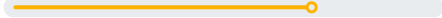
This test aims to measure your vocabulary, grammar and reading comprehension skills.

You have a fairly rich vocabulary and a strong command of English grammar. You are able to read and understand complex text. Having a good command of the English language is important to communicate with internal stakeholders and clients, as well as to interpret reports, articles and complex texts at work.

Logical Ability

 69 / 100

Inductive Reasoning

 70 / 100

This competency aims to measure the your ability to synthesize information and derive conclusions.

It is commendable that you have excellent inductive reasoning skills. You are able to make specific observations to generalize situations and also formulate new generic rules from variable data.



Deductive Reasoning

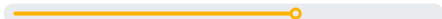
 71 / 100

This competency aims to measure the your ability to synthesize information and derive conclusions.

It is commendable that you have excellent inductive reasoning skills. You are able to make specific observations to generalize situations and also formulate new generic rules from variable data.



Abductive Reasoning

 66 / 100

Quantitative Ability (Advanced)

 58 / 100

This test aims to measure your ability to solve problems on basic arithmetic operations, probability, permutations and combinations, and other advanced concepts.

You are able to solve word problems on basic concepts of percentages, ratio, proportion, interest, time and work. Having a strong hold on these concepts can help you understand the concept of work efficiency and how interest is accrued on bank savings. It can also guide you in time management, work planning, and resource allocation in complex projects.

Personality

Competencies



Extraversion



Extraversion refers to a person's inclination to prefer social interaction over spending time alone. Individuals with high levels of extraversion are perceived to be outgoing, warm and socially confident.

- You are outgoing and seek out opportunities to meet new people.
- You tend to enjoy social gatherings and feels comfortable amongst strangers and friends equally.
- You display high energy levels and like to indulge in thrilling and exciting activities.
- You may tend to be assertive about your opinions and prefer action over contemplation.
- You take initiative and are more inclined to take charge than to wait for others to lead the way.
- Your personality is well suited for jobs demanding frequent interaction with people.



Conscientiousness



Conscientiousness is the tendency to be organized, hard working and responsible in one's approach to your work. Individuals with high levels of this personality trait are more likely to be ambitious and tend to be goal-oriented and focused.

- You are flexible and able to adapt your work pace to the job at hand.
- You are usually spontaneous but you are likely to stick to a plan whenever necessary.
- You tend to be cautious when you deem it necessary.
- You may prefer to act according to the rules.
- You are confident in your ability to achieve goals but may need support to overcome occasional setbacks.
- You are an efficient worker and try to perform better than your peers. You are well suited for jobs allowing flexibility regarding operating procedures.



Agreeableness



Agreeableness refers to an individual's tendency to be cooperative with others and it defines your approach to interpersonal relationships. People with high levels of this personality trait tend to be more considerate of people around them and are more likely to work effectively in a team.

- You are considerate and sensitive to the needs of others.
- You tend to put the needs of others ahead of your own.
- You are likely to trust others easily without doubting their intentions.
- You are compassionate and may be strongly affected by the plight of both friends and strangers.
- You are humble and modest and prefer not to talk about personal accomplishments.
- Your personality is more suitable for jobs demanding cooperation among employees.



Openness to Experience



Openness to experience refers to a person's inclination to explore beyond conventional boundaries in different aspects of life. Individuals with high levels of this personality trait tend to be more curious, creative and innovative in nature.

- You tend to be curious in nature and is generally open to trying new things outside your comfort zone.
- You may have a different approach to solving conventional problems and tend to experiment with those solutions.
- You are creative and tends to appreciate different forms of art.
- You are likely to be in touch with your emotions and is quite expressive.
- Your personality is more suited for jobs requiring creativity and an innovative approach to problem solving.



Emotional Stability



Emotional stability refers to the ability to withstand stress, handle adversity, and remain calm and composed when working through challenging situations. People with high levels of this personality trait tend to be more in control of their emotions and are likely to perform consistently despite difficult or unfavourable conditions.

- You are calm and composed in nature.
- You tend to maintain composure during high pressure situations.
- You are very confident and comfortable being yourself.
- You find it easy to resist temptations and practice moderation.
- You are likely to remain emotionally stable in jobs with high stress levels.



Polychronicity



Polychronicity refers to a person's inclination to multitask. It is the extent to which the person prefers to engage in more than one task at a time and believes that such an approach is highly productive. While this trait describes the personality disposition of a person to multitask, it does not gauge their ability to do so successfully.

- You pursue multiple tasks simultaneously, switching between them when needed.
- You prefer working to achieve some progress on multiple tasks simultaneously than completing one task before moving on to the next task.
- You tend to believe that multitasking is an efficient way of doing things and prefers an action packed work life with multiple projects.

3 | Response

Automata



24 / 100

[Code Replay](#)

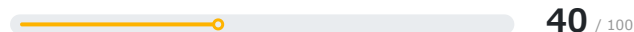
Question 1 (Language: Java)

A company wants to provide a cab service for their N employees. The employees have IDs ranging from 0 to N-1. The company has calculated the total distance from an employee's residence to the company, considering the path to be followed by the cab to be a straight path. The distance to the company from itself is 0. The distance for the employees who live to the left side of the company is represented with a negative sign. The distance for the employees who live to the right side of the company is represented with a positive sign. The cab will be allotted a range of distance. The company wants to find the distance for the employees who live within the particular distance range.

Write an algorithm to find the distance for the employees who live within the distance range.

Scores

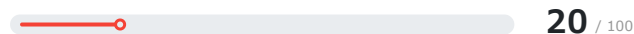
Programming Ability



40 / 100

Emerging basic structure. Appropriate keywords and tokens present, showing some understanding of a part of the problem.

Functional Correctness



20 / 100

The source code does not pass any basic test cases. It is either due to incorrect logic or runtime errors. Some advanced or edge cases may randomly pass.

Programming Practices



75 / 100

Low readability, high on program structure. The source code does not follow best practices in its formatting and may contain a few redundant/improper coding constructs.

Final Code Submitted

Compilation Status: Pass

```
1 import java.util.*;
2 import java.lang.*;
3 import java.io.*;
4
5 /*
6 *
7 */
8 public class Solution
9 {
10     public static int[] funcDistance(int[] inputDist, int start, int
        end)
11     {
12         int[] answer = new int[100];
13         // Write your code here
14         int n=inputDist.length;
15         int j=0;
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code: $O(N \log N)$

*N represents size of the list

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilities and Errors


```

16
17     for (int i=0;i<n-1;i++){
18         if(inputDist[i]>=start && inputDist[i]<=end){
19             answer[j]=inputDist[i];
20             j++;
21         }
22         else{
23             continue;
24         }
25     }
26     return answer;
27 }
28
29 public static void main(String[] args)
30 {
31     Scanner in = new Scanner(System.in);
32     //input for inputDist
33     int inputDist_size = in.nextInt();
34     int inputDist[] = new int[inputDist_size];
35     for(int idx = 0; idx < inputDist_size; idx++)
36     {
37         inputDist[idx] = in.nextInt();
38     }
39     // input for start
40     int start = in.nextInt();
41
42     // input for end
43     int end = in.nextInt();
44
45
46     int[] result = funcDistance(inputDist, start, end);
47     for(int idx = 0; idx < result.length - 1; idx++)
48     {
49         System.out.print(result[idx] + " ");
50     }
51     System.out.print(result[result.length - 1]);
52 }
53 }
54

```

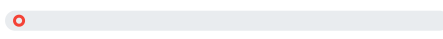
Readability & Language Best Practices

Line 14,15: Variables are given very short names.

Test Case Execution

Passed TC: 0%

Total score

 0/12

0%

Basic(0/6)

0%

Advance(0/6)

0%

Edge(0/0)

Compilation Statistics

8

Total attempts

8

Successful

0

Compilation errors

0

Sample failed

0

Timed out

7

Runtime errors

Response time:

00:23:37

Average time taken between two compile attempts:

00:02:57

Average test case pass percentage per compile:

0%

i Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

i Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

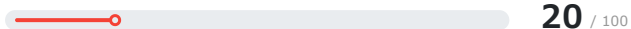
Question 2 (Language: Java)

Given a route in a straight line, N buses operate between various bus stations. There is a workstation at the start of the route. The distances of the bus stations from the workstation are calculated. The transportation authority wants to decrease the number of buses that it operates in the city. If any buses are found to have overlapping routes, then these buses will be replaced by a single bus. The authority wants to determine how many buses will remain after the buses with overlapping routes have been eliminated.

Write an algorithm to find how many buses will remain after the buses with overlapping routes have been eliminated.

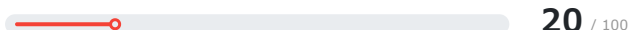
Scores

Programming Ability



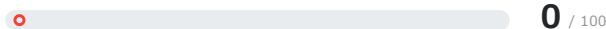
Code seems to be unrelated to the given problem.

Functional Correctness



The source code does not pass any basic test cases. It is either due to incorrect logic or runtime errors. Some advanced or edge cases may randomly pass.

Programming Practices



Programming practices score cannot be generated. This is because source code has syntax/runtime errors and is unparseable or the source code does not meet the minimum code-length specifications.

Final Code Submitted

Compilation Status: Pass

```

1 import java.util.*;
2 import java.lang.*;
3 import java.io.*;
4
5 /*
6 *
7 */
8 public class Solution
9 {
10     public static int busRemaining(int[][] busStation)
11     {
12         int answer = 0;
13         // Write your code here
14         int n=busStation.length;
15
16
17         return answer;
18     }
19
20     public static void main(String[] args)
21     {
22         Scanner in = new Scanner(System.in);
23         // input for busStation
24         int busStation_row = in.nextInt();
25         int busStation_col = in.nextInt();
26         int busStation[][] = new int[busStation_row][busStation_c
27         ol];
28         for(int idx = 0; idx < busStation_row; idx++)
29         {
30             for(int jdx = 0; jdx < busStation_col; jdx++)
31             {
32                 busStation[idx][jdx] = in.nextInt();
33             }
34         }
35     }
36 }
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code: $O(N)$

*N represents number of buses

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilities and Errors

There are no errors in the candidate's code.

```

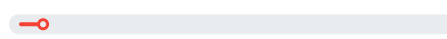
34
35     int result = busRemaining(busStation);
36     System.out.print(result);
37
38 }
39 }
40

```

Test Case Execution

Passed TC: 4.55%

Total score

 1/22

0%

Basic(0/12)

0%

Advance(0/7)

33%

Edge(1/3)

Compilation Statistics

0

Total attempts

0

Successful

0

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:19:06

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Automata Fix



15 / 100

[Code Replay](#)

Question 1 (Language: Java)

The function/method **selectionSortArray** performs an in-place selection sort on the given input list which will be sorted in ascending order.

The function/method **selectionSortArray** accepts two arguments - *len*, an integer representing the length of the input list and *arr*, a list of integers representing the input list, respectively.

The function/method **selectionSortArray** compiles successfully but fails to get the desired result for some test cases due to logical errors. Your task is to fix the code so that it passes all the test cases.

Note:

In this particular implementation of selection sort, the smallest element in the list is swapped with the element at first index, the next smallest element is swapped with the element at the next index and so on.

Scores**Final Code Submitted****Compilation Status: Pass**

```
1 // You can print the values to stdout for debugging
2 class Solution
3 {
4     public void selectionSortArray( int len, int[] arr )
5     {
6         int x = 0 , y = 0 ;
7         for( x = 0 ; x < len ; x ++ )
8         {
9             int index_of_min = x;
10            for( y = x ; y < len ; y ++ )
11            {
12                if(arr[index_of_min]>arr[y])
13                {
14                    index_of_min = y;
15                }
16            }
17            int temp = arr[x];
18            arr[x] = arr[index_of_min];
19            arr[index_of_min] = temp;
20        }
21    }
22 }
```

Code Analysis**Average-case Time Complexity**

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

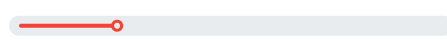
There are no errors in the candidate's code.

Structural Vulnerabilities and Errors

There are no errors in the candidate's code.

Test Case Execution**Passed TC: 22.22%**

Total score

 2/9**50%**

Basic(2/4)

0%

Advance(0/5)

0%

Edge(0/0)

Compilation Statistics

0

Total attempts

0

Successful

0

Compilation errors

1

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:01:12

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

22.2%

i Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

i Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 2 (Language: Java)

You are given a predefined structure `Point` and also a collection of related functions/methods that can be used to perform some basic operations on the structure.

You must implement the function/method ***isTriangle*** which accepts three points *P1*, *P2*, *P3* as inputs and checks whether the given three points form a triangle.

If they form a triangle, the function/method returns an integer 1. Otherwise, it returns an integer 0.

.

Helper Description

The following class is used to represent a point and is already implemented in the default code (Do not write this definition again in your code):

```
public class Point
```

```
{
```

```

public int x, y;

public Point()

{}

public Point(int x1, int y1)

{

    x = x1 ;

    y = y1 ;

}

public double calculateDistance(Point P)

{

    /*Return the euclidean distance between two input points.

    This can be called as -

    * If P1 and P2 are two points then -

    * P1.calculateDistance(P2);*/

}

}

```

Scores

Final Code Submitted

Compilation Status: Fail

```

1 // You can print the values to stdout for debugging
2 class Solution
3 {
4     int isTriangle(Point P1, Point P2, Point P3)
5     {
6         // write your code here
7         if(Point P1<(Point P2 + Point P3) && Point P2<(Point P1 +
Point P3) && Point P3<(Point P2 + Point P1)){
8             return 1;
9         }
10        else{
11            return 0;
12        }
13    }
14
15 }

```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

Solution.java:7: error: ')' expected
if(Point P1<(Point P2 + Point P3) && Point P2<(Point P1 + Point P3) && Point P3<(Point P2 + Point P1)){
^

Solution.java:7: error: illegal start of type
if(Point P1<(Point P2 + Point P3) && Point P2<(Point P1 + Point P3) && Point P3<(Point P2 + Point P1)){

^

Solution.java:7: error: ';' expected
 if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: variable declaration not
 allowed here

if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: not a statement

if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: ';' expected

if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: not a statement

if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: ';' expected

if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: ';' expected

if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: illegal start of type

if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: illegal start of expression

if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: ';' expected

if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: not a statement

if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: ';' expected

if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: not a statement

if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: ';' expected

if(Point P1<(Point P2 + Point P3) && Point P2<(Point
 P1 + Point P3) && Point P3<(Point P2 + Point P1)){
 ^

Solution.java:7: error: ';' expected


```
if(Point P1<(Point P2 + Point P3) && Point P2<(Point
P1 + Point P3) && Point P3<(Point P2 + Point P1)){
^
Solution.java:7: error: illegal start of type
if(Point P1<(Point P2 + Point P3) && Point P2<(Point
P1 + Point P3) && Point P3<(Point P2 + Point P1)){
^
Solution.java:7: error: illegal start of expression
if(Point P1<(Point P2 + Point P3) && Point P2<(Point
P1 + Point P3) && Point P3<(Point P2 + Point P1)){
^
Solution.java:7: error: ';' expected
if(Point P1<(Point P2 + Point P3) && Point P2<(Point
P1 + Point P3) && Point P3<(Point P2 + Point P1)){
^
Solution.java:7: error: not a statement
if(Point P1<(Point P2 + Point P3) && Point P2<(Point
P1 + Point P3) && Point P3<(Point P2 + Point P1)){
^
Solution.java:7: error: ';' expected
if(Point P1<(Point P2 + Point P3) && Point P2<(Point
P1 + Point P3) && Point P3<(Point P2 + Point P1)){
^
Solution.java:7: error: not a statement
if(Point P1<(Point P2 + Point P3) && Point P2<(Point
P1 + Point P3) && Point P3<(Point P2 + Point P1)){
^
Solution.java:7: error: ';' expected
if(Point P1<(Point P2 + Point P3) && Point P2<(Point
P1 + Point P3) && Point P3<(Point P2 + Point P1)){
^
Solution.java:10: error: 'else' without 'if'
else{
^
25 errors
```

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Compilation Statistics

6

Total attempts

0

Successful

6

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:	00:07:52
Average time taken between two compile attempts:	00:01:19
Average test case pass percentage per compile:	0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 3 (Language: Java)

The function/method ***patternPrint*** accepts an argument *num*, an integer.
The function/method ***patternPrint*** prints *num* lines in the following pattern.

For example, *num* = 4, the pattern should be:

```
1
1 1
1 1 1
1 1 1 1
```

The function/method ***patternPrint*** compiles successfully but fails to print the desired result for some test cases due to incorrect implementation of the function/method. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted	Compilation Status: Pass	Code Analysis
<pre> 1 // You can print the values to stdout for debugging 2 class Solution 3 { 4 void patternPrint(int num) 5 { 6 int print=1; 7 for(int i=0;i<num;i++) 8 { 9 for(int j=0;j<=i+1;j++); 10 { 11 System.out.print(print+" "); 12 } 13 System.out.println(); </pre>		Average-case Time Complexity Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases. Best case code: *N represents
		Errors/Warnings There are no errors in the candidate's code.

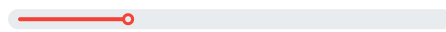
```
14    }  
15  }  
16 }  
17
```

Structural Vulnerabilities and Errors

There are no errors in the candidate's code.

Test Case ExecutionPassed TC: **25%**

Total score

 2/8**14%**

Basic(1/7)

0%

Advance(0/0)

100%

Edge(1/1)

Compilation Statistics**3**

Total attempts

3

Successful

0

Compilation errors

3

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:03:48

Average time taken between two compile attempts:

00:01:16

Average test case pass percentage per compile:

12.5%**i Average-case Time Complexity**

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

i Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 4 (Language: Java)

The function/method ***allExponent*** returns a real number representing the result of exponentiation of base raised to power exponent for all input values. The function/method ***allExponent*** accepts two arguments - *baseValue*, an integer representing the base and *exponentValue*, an integer representing the exponent.

The incomplete code in the function/method ***allExponent*** works only for positive values of the exponent. You must complete the code and make it work for negative values of exponent as well.

Another function/method **positiveExponent** uses an efficient way for exponentiation but accepts only positive *exponent* values. You are supposed to use this function/method to complete the code in **allExponent** function/method.

Helper Description

The following class is used to represent a Exponent and is already implemented in the default code (Do not write this definition again in your code):

```
public class Exponent
{
    public int base;

    public int exponent;

    int positiveExponent()
    {
        /*It calculate the Exponent for positive value of exponentValue

        This can be called as -

        Exponent exp = new Exponent(baseValue, exponentValue);

        float res = exp.positiveExponent();*/
    }
}
```

Scores

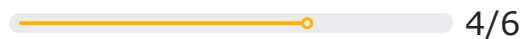
Final Code Submitted	Compilation Status: Pass	Code Analysis
<pre> 1 // You can print the values to stdout for debugging 2 class Solution 3 { 4 float allExponent(int baseValue, int exponentValue) 5 { 6 float res = 1; 7 if(exponentValue >=0) 8 { 9 Exponent exp = new Exponent(baseValue, exponentValue); 10 res = (float)exp.positiveExponent(); 11 } 12 else 13 { 14 // write your code here for negative exponentInput 15 } 16 } 17 return res; </pre>		Average-case Time Complexity
		Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases. Best case code: *N represents
		Errors/Warnings
		There are no errors in the candidate's code.
		Structural Vulnerabilites and Errors
		There are no errors in the candidate's code.

```
18 }  
19 }
```

Test Case Execution

Passed TC: 66.67%

Total score



4/6

100%

Basic(2/2)

33%

Advance(1/3)

100%

Edge(1/1)

Compilation Statistics

1

Total attempts

0

Successful

1

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:02:00

Average time taken between two compile attempts:

00:02:00

Average test case pass percentage per compile:

0%

i Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

i Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 5 (Language: Java)

Lisa always forgets her birthday which is on the 5th of July. So, develop a function/method which will be helpful to remember her birthday.

The function/method **checkBirthDay** return an integer '1' if it is her birthday else returns 0. The function/method **checkBirthDay** accepts two arguments - *month*, a string representing the month of her birthday and *day*, an integer representing the date of her birthday.

The function/method **checkBirthDay** compiles successfully but fails to return the desired result for some test cases. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted

Compilation Status: Pass

```
1 // You can print the values to stdout for debugging
2 class Solution
3 {
4     int checkBirthDay(String month, int day)
5     {
6         if(!(month.equals("July")) && (day==5))
7             return 1;
8         else
9             return 0;
10    }
11 }
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

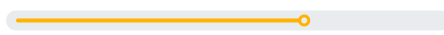
Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **66.67%**

Total score

 6/9**67%**

Basic(4/6)

67%

Advance(2/3)

0%

Edge(0/0)

Compilation Statistics

1

Total attempts

1

Successful

0

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:02:02

Average time taken between two compile attempts:

00:02:02

Average test case pass percentage per compile:

66.7%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 6 (Language: Java)

The function/method ***descendingSortArray*** performs an in-place sort on the given input list which will be sorted in descending order.

The function/method ***descendingSortArray*** accepts two arguments - *len*, an integer representing the length of the input list and *arr*, a list of integers representing the input list, respectively.

5

The function/method ***descendingSortArray*** compiles successfully but fails to get the desired result for some test cases due to logical errors. Your task is to fix the code so that it passes all the test cases.

Scores

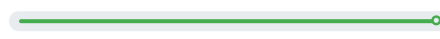
Final Code Submitted	Compilation Status: Pass	Code Analysis
<pre> 1 // You can print the values to stdout for debugging 2 class Solution 3 { 4 public void descendingSortArray(int len, int[] arr) 5 { 6 int small, pos, i, j, temp; 7 for(i = 0 ; i <= len - 1 ; i ++) 8 { 9 for(j = i ; j < len ; j ++) 10 { 11 temp = 0; 12 if(arr[i] < arr[j]) 13 { 14 temp = arr[i]; 15 arr[i] = arr[j]; 16 arr[j] = temp; 17 } </pre>		Average-case Time Complexity Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases. Best case code: *N represents
		Errors/Warnings There are no errors in the candidate's code.
		Structural Vulnerabilities and Errors There are no errors in the candidate's code.

```
18     }  
19     }  
20 }  
21 }
```

Test Case Execution

Passed TC: 100%

Total score

 10/10

100%

Basic(6/6)

100%

Advance(4/4)

0%

Edge(0/0)

Compilation Statistics

1

Total attempts

1

Successful

0

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:02:02

Average time taken between two compile attempts:

00:02:02

Average test case pass percentage per compile:

100%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 7 (Language: Java)

The function/method **matrixSum** returns an integer representing the sum of elements of the input matrix. The function/method **matrixSum** accepts three arguments - *rows*, an integer representing the number of rows of the input matrix, *columns*, an integer representing the number of columns of the input matrix and *matrix*, a two-dimensional array representing the input matrix.

The function/method **matrixSum** compiles unsuccessfully due to syntactical error. Your task is to debug the program so that it passes all test cases.

Scores

Final Code Submitted

Compilation Status: Fail

```
1 // You can print the values to stdout for debugging
2 class Solution
3 {
4     int matrixSum(int rows, int columns, int[][] matrix)
5     {
6         int i, j, sum=0;
7         for(i=0;i<rows;i++)
8         {
9             for(j=0;j<columns;j++)
10                sum + = matrix(i)(j);
11        }
12        return sum;
13    }
14 }
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

Solution.java:10: error: illegal start of expression
sum + = matrix(i)(j);
^

Solution.java:10: error: ';' expected
sum + = matrix(i)(j);
^

Solution.java:10: error: not a statement
sum + = matrix(i)(j);
^

Solution.java:10: error: ';' expected
sum + = matrix(i)(j);
^

4 errors

Structural Vulnerabilities and Errors

There are no errors in the candidate's code.

Compilation Statistics

0

Total attempts

0

Successful

0

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:00:39

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

WriteX - Essay Writing



86 / 100

CEFR: C1

Question

Nowadays, many youngsters constantly look for job opportunities abroad and in the long term want to settle there.

In your view, what are the reasons for the same? Do you wish to settle abroad or stay back in India? Substantiate your response with reasons.

Scores

Content Score

Grammar Score

 87 / 100

 84 / 100

Response

Nowadays, many youngsters constantly look for job opportunities abroad and in the long term want to settle there. This has been increasing with time. There can be a lot of reasons a person wants to settle abroad leaving their country. The general reasons I feel happen are the rapid development in the foreign countries. Also there is a lot of foreign impact on our country through social media, series and movies. People tend to like the way of living for foreigners more than their own country's. Many countries are ahead of our country in many ways especially in terms of economy and growth. What people earn outside is usually more than what they can earn in India. The working environment also I feel is less hectic and less toxic. Recently the value of Indian currency has fell down a lot resulting in people to think that they can live a more wealthy and happy life outside India. I also wish to visit other countries but do not tend to settle there. I feel my motherland is more comfortable for me always. I want to go abroad for experience. Even I want to stay in different countries for few months but not settle permanently. I feel the comfort and sooth one can feel in their own country cannot be felt somewhere else. But settling abroad is not a bad thing in my opinion. I have seen that people of India living abroad have gained a more wealthy life in

Error Summary

	Spelling	9
	White Space	0
	Style	1
	Grammar	21
	Typographical	0

majority of cases. Living abroad gives you a huge amount of exposure to the outside world, different type of people and places. But, it can be a bit difficult also for many people to adjust to an entirely new environment with new people and culture. I feel every person should go outside their country and explore how the world works.

Essay Statistics

309	20	15	166	125
Total words	Total sentences	Average sentence length	Total unique words	Total stop words

Error Details

Spelling

...ral reasons I feel happen are the rapid development in the foreign countries. Also there is...	Possible spelling mistake found
... way of living for foreigners more than their own country's. Many countries are ahead...	Possible spelling mistake found
...tside India. I also wish to visit other countries but do not tend to settle there. I feel ...	Possible spelling mistake found
...wish to visit other countries but do not tend to settle there. I feel my mother land ...	Possible spelling mistake found
settle there. I feel my mother land is more comfortable for	Possible spelling mistake found. Consider replacing the highlighted text with: 'motherland'.
sooth one can feel in there own country cannot be	Possible spelling mistake found. Consider replacing the highlighted text with: 'their'.
...road have gained a more wealthy life in majority of cases. Living abroad gives you a hug...	Possible spelling mistake found
...any people to adjust to an entirely new environment with new people and culture. I feel eve...	Possible spelling mistake found
... culture. I feel every person should go outside their country and explore how the world...	Possible spelling mistake found
...e. I feel every person should go outside their country and explore how the world works...	Possible spelling mistake found

Style

...land is more comfortable for me always. I want to go a broad for experience. Even ...	Three successive sentences begin with the same word. Consider rewording the sentence or use a thesaurus to find a synonym.
--	--

Grammar

Nowadays, many youngsters constantly look for job opportunities abroad and in the long term want to settle there.	Possible grammar error found. Consider replacing it with "term,".
---	---

There can be a lot of reasons a person wants to settle **abroad** leaving their country.

Possible grammar error found. Consider replacing it with "abroad,".

There can be a lot of reasons a person wants to settle **abroad** **leaving** their country.

Possible grammar error found. Consider replacing it with "leave".

The general reasons I feel happen are the rapid development in **the** foreign countries.

Possible grammar error found. Consider removing "the" from here.

People tend to like the way of living **for** foreigners more than their own country's.

Possible grammar error found. Consider replacing it with "of".

People tend to like the way of living for foreigners more than their own **country's**.

Possible grammar error found. Consider replacing it with "country".

Many countries are ahead of our country in many **ways** especially in terms of economy and growth.

Possible grammar error found. Consider replacing it with "ways,".

The working environment also **I** feel is less hectic and less toxic.

Possible grammar error found. Consider removing "I" from here.

The working environment also **I feel** is less hectic and less toxic.

Possible grammar error found. Consider replacing it with "feels".

Recently the value of **indian** currency has fell down a lot resulting in people to think that they can live a more wealthy and happy life outside India.

Possible grammar error found. Consider inserting "the" over here.

Recently the value of indian currency has **fell** down a lot resulting in people to think that they can live a more wealthy and happy life outside India.

Possible grammar error found. Consider replacing it with "fallen".

Recently the value of indian currency has fell down a **lot** resulting in people to think that they can live a more wealthy and happy life outside India.

Possible grammar error found. Consider replacing it with "lot,".

Recently the value of indian currency has fell down a lot resulting in people **to think** that they can live a more wealthy and happy life outside India.

Possible grammar error found. Consider replacing it with "thinking".

I feel my mother land is more comfortable for me always.

Possible grammar error found. Consider inserting "always" over here.

I feel my mother land is more comfortable for me **always**.

Possible grammar error found. Consider removing "always." from here.

Even I want to stay in different countries for few months but not settle permanently.

Possible grammar error found. Consider removing "Even" from here.

Even I want to stay in different countries for **few** months but not settle permanently.

Possible grammar error found. Consider inserting "a" over here.

But settling abroad is not a bad **thing** in my opinion.

Possible grammar error found. Consider replacing it with "thing,".

I have seen that people **of** India living abroad have gained a more wealthy life in majority of cases.

Possible grammar error found. Consider replacing it with "in".

But, it can be **be a** bit difficult also for many people to adjust to an entirely new environment with new people and culture.

Possible grammar error found. Consider inserting "also" over here.

But, it can be a bit difficult **also** for many people to adjust to an entirely new environment with new people and culture.

Possible grammar error found. Consider removing "also" from here.

4 | Learning Resources

English Comprehension

[Read novels to enhance your comprehension skills](#)

[Read opinions to improve your comprehension](#)

[Read research papers online](#)


Logical Ability

[Play chess and challenge your reasoning skills](#)

[Practice inductive reasoning on visual data](#)

[Learn about pattern identification](#)


Quantitative Ability (Advanced)

[Watch a video on the history of algebra and its applications](#)

[Learn about proportions and its practical usage](#)

[Learn about calculating percentages manually](#)


Icon Index



Free Tutorial



Paid Tutorial



Youtube Video



Web Source



Wikipedia



Text Tutorial



Video Tutorial



Google Playstore